ONLINE BOOKSTORE
SQL DATA ANALYSIS PROJECT

Database: OnlineBookstore
Technology: PostgreSQL / pgAdmin 4
Dataset: Books, Customers and Orders CSV files

Prepared By : Khushi Chaudhary

Query 1: Retrieve all books in the 'Fiction' genre

SQL Solution

```
SELECT * FROM Books
WHERE Genre='Fiction';
```

Explanation

Filters the Books table to show only books whose Genre is Fiction.

Query Output

Rows returned: **60**. Preview below shows the first 8 rows.

Book_ID	Title	Author	Genre	Published_Year	Price	Stock
4	Customizable 24hour product	Christopher Andrews	Fiction	2020	43.52	8
22	Multi-layered optimizing migration	Wesley Escobar	Fiction	1908	39.23	78
28	Expanded analyzing portal	Lisa Coffey	Fiction	1941	37.51	79
29	Quality-focused multi-tasking challenge	Katrina Underwood	Fiction	1905	31.12	100
31	Implemented encompassing conglomeration	Melissa Taylor	Fiction	2010	21.23	44
39	Optimized national process improvement	Megan Goodwin	Fiction	1978	10.99	42
40	Adaptive didactic interface	Natalie Gonzalez	Fiction	1923	25.97	94
47	Reverse-engineered directional conglomeration	John Christian	Fiction	2006	20.37	90

Query 2: Find books published after the year 1950

SQL Solution

```
SELECT * FROM Books
WHERE published_year>1950;
```

Explanation

Returns books with a Published_Year greater than 1950.

Query Output

Rows returned: **292**. Preview below shows the first 8 rows.

Book_ID	Title	Author	Genre	Published_Year	Price	Stock
2	Persevering reciprocal knowledge user	Mario Moore	Fantasy	1971	35.8	19
4	Customizable 24hour product	Christopher Andrews	Fiction	2020	43.52	8
5	Adaptive 5thgeneration encoding	Juan Miller	Fantasy	1956	10.95	16
6	Advanced encompassing implementation	Bryan Morgan	Biography	1985	6.56	2
8	Persistent local encoding	Troy Cox	Science Fiction	2019	48.99	84
9	Optimized interactive challenge	Colin Buckley	Fantasy	1987	14.33	70
10	Ergonomic national hub	Samantha Ruiz	Mystery	2015	24.63	25
11	Secured zero tolerance time-frame	Denise Barnes	Fantasy	1998	35.95	10

Query 3: List all the customers from the canada

SQL Solution

```
SELECT * FROM Customers
WHERE country='Canada';
```

Explanation

Finds customers whose Country value is Canada.

Query Output

Rows returned: **3**. All returned rows are shown below.

Customer_ID	Name	Email	Phone	City	Country
38	Nicholas Harris	christine93@perkins.com	1234567928	Davistown	Canada
415	James Ramirez	robert54@hall.com	1234568305	Maxwelltown	Canada
468	David Hart	stokesrebecca@gmail.com	1234568358	Thompsonfurt	Canada

Query 4: Show orders a placed in november 2023

SQL Solution

```
SELECT * FROM Orders
WHERE order_date BETWEEN '2023-11-01'AND'2023-11-30';
```

Explanation

Returns orders dated from 1 November 2023 through 30 November 2023, inclusive.

Query Output

Rows returned: **25**. Preview below shows the first 8 rows.

Order_ID	Customer_ID	Book_ID	Order_Date	Quantity	Total_Amount
4	433	343	2023-11-25	7	301.21
19	496	60	2023-11-17	9	316.26
75	291	375	2023-11-30	5	170.75
132	469	333	2023-11-22	7	194.32
137	474	471	2023-11-25	8	363.04
163	207	384	2023-11-23	3	101.76
182	129	293	2023-11-01	7	125.51
200	313	303	2023-11-23	1	6.57

Query 5: Retrieve the total stock of books available

SQL Solution

```
SELECT SUM(stock)as Total_Stock
FROM Books;
```

Explanation

Adds the Stock values for all books to calculate total recorded inventory.

Query Output

Rows returned: **1**. All returned rows are shown below.

Total_Stock
25056

Query 6: Find the details of the most expensive book

SQL Solution

```
SELECT * FROM Books
ORDER BY price DESC LIMIT 1;
```

Explanation

Sorts books by Price from highest to lowest and returns the first row.

Query Output

Rows returned: **1**. All returned rows are shown below.

Book_ID	Title	Author	Genre	Published_Year	Price	Stock
340	Proactive system-worthy orchestration	Robert Scott	Mystery	1907	49.98	88

Query 7: Show all customers who ordered more than 1 quantity of a book

SQL Solution

```
SELECT * FROM Orders
WHERE quantity>1;
```

Explanation

Shows order records where the ordered Quantity is greater than 1.

Query Output

Rows returned: **438**. Preview below shows the first 8 rows.

Order_ID	Customer_ID	Book_ID	Order_Date	Quantity	Total_Amount
1	84	169	2023-05-26	8	188.56
2	137	301	2023-01-23	10	216.6
3	216	261	2024-05-27	6	85.5
4	433	343	2023-11-25	7	301.21
5	14	431	2023-07-26	7	136.36
6	439	119	2024-10-11	5	249.4
7	195	467	2023-10-23	6	82.92
8	32	159	2024-05-07	4	144.84

Query 8: Retrieve all the orders where the total amount exceeds $20

SQL Solution

```
SELECT * FROM Orders
WHERE total_amount>20;
```

Explanation

Returns orders whose Total_Amount is greater than 20.

Query Output

Rows returned: **473**. Preview below shows the first 8 rows.

Order_ID	Customer_ID	Book_ID	Order_Date	Quantity	Total_Amount
1	84	169	2023-05-26	8	188.56
2	137	301	2023-01-23	10	216.6
3	216	261	2024-05-27	6	85.5
4	433	343	2023-11-25	7	301.21
5	14	431	2023-07-26	7	136.36
6	439	119	2024-10-11	5	249.4
7	195	467	2023-10-23	6	82.92
8	32	159	2024-05-07	4	144.84

Query 9: List all genres available in the Books table

.

SQL Solution

```
SELECT DISTINCT genre FROM Books;
```

Explanation

Lists each distinct genre appearing in the Books table.

Query Output

Rows returned: **7**. All returned rows are shown below.

Genre
Biography
Fantasy
Non-Fiction
Fiction
Romance
Science Fiction
Mystery

Query 10: Find the books with the lowest stock

SQL Solution

```
SELECT * FROM Books
ORDER BY stock;
```

Explanation

Sorts all books by Stock in ascending order, placing the lowest-stock books first.

Query Output

Rows returned: **500**. Preview below shows the first 8 rows.

Book_ID	Title	Author	Genre	Published_Year	Price	Stock
44	Networked systemic implementation	Ryan Frank	Science Fiction	1965	13.55	0
60	Robust eco-centric capacity	Brian Haney	Biography	1990	35.14	0
127	Business-focused real-time benchmark	David Nelson	Science Fiction	1997	11.66	0
163	Object-based eco-centric challenge	Douglas Mccarthy	Non-Fiction	1905	19.11	0
378	Future-proofed heuristic function	Samantha Mcclain	Romance	1903	6.01	0
19	Progressive asymmetric Internet solution	Sean Miller	Science Fiction	1990	11.31	1
137	Networked contextually-based encryption	Leslie Montoya	Biography	1942	26.33	1
199	Configurable fault-tolerant interface	Mark Parker	Mystery	1923	42.81	1

Query 11: Calculate the total revenue generated from all orders

SQL Solution

```
SELECT SUM(total_amount)AS Revenue
FROM Orders;
```

Explanation

Adds Total_Amount across all order records to calculate total recorded revenue.

Query Output

Rows returned: **1**. All returned rows are shown below.

Revenue
75628.66

Query 12: Retrieve the total number of books sold for each genre

SQL Solution

```
SELECT B.Genre,SUM(o.Quantity)AS Total_Books_sold
FROM Orders o Join Books b
ON o.book_id=b.book_id
GROUP BY b.Genre;
```

Explanation

Joins orders to books and sums the quantity sold for each genre.

Query Output

Rows returned: **7**. All returned rows are shown below.

Genre	Total_Books_sold
Biography	285
Fantasy	446
Fiction	225
Mystery	504
Non-Fiction	351
Romance	439
Science Fiction	447

Query 13: Find the average price of books in the 'Fantasy'genre

SQL Solution

```
SELECT AVG(price)AS Average_Price
FROM Books
WHERE Genre='Fantasy';
```

Explanation

Calculates the average Price for books in the Fantasy genre.

Query Output

Rows returned: **1**. All returned rows are shown below.

Average_Price
25.98

Query 14: List customers who have placed at least 2 orders

SQL Solution

```
SELECT o.customer_id,c.name,COUNT(o.Order_id)AS ORDER_COUNT
FROM orders o
JOIN customers c ON o.customer_id=c.customer_id
GROUP BY o.customer_id ,c.name
HAVING COUNT (Order_id)>=2;
```

Explanation

Joins orders to customers, groups by customer, and keeps customers with at least two order records.

Query Output

Rows returned: **139**. Preview below shows the first 8 rows.

Customer_ID	Name	ORDER_COUNT
2	Crystal Clements	2
6	Stephen Vasquez	2
8	Matthew Johnson	2
13	Kristine Kim	2
14	John Wood	2
15	Vanessa Gaines	2
16	Stacey Flores	3
21	Edgar Frost	2

Query 15: Find the most frequency ordered book

SQL Solution

```
SELECT O.Book_id,B.title,COUNT(O.order_id)AS ORDER_COUNT
FROM orders O
JOIN Books B ON O.Book_id=B.Book_id
GROUP BY O.Book_id,B.title
ORDER BY ORDER_COUNT DESC;
```

Explanation

Counts order records for each book and sorts from most frequently ordered to least. The query returns all books with orders, not only the single top book.

Query Output

Rows returned: **317**. Preview below shows the first 8 rows.

Book_ID	Title	ORDER_COUNT
31	Implemented encompassing conglomeration	4
73	Realigned multi-tasking installation	4
88	Robust tangible hardware	4
120	Integrated secondary access	4
273	Devolved zero administration process improvement	4
333	Advanced responsive extranet	4
491	Pre-emptive intangible adapter	4
52	Triple-buffered neutral adapter	3

Query 16: Show the top 3 most expensive book of 'Fantasy'Genre

SQL Solution

```
SELECT*FROM Books
WHERE Genre='Fantasy'
ORDER BY price DESC LIMIT 3;
```

Explanation

Filters to Fantasy books, sorts by price descending, and returns the three most expensive rows.

Query Output

Rows returned: **3**. All returned rows are shown below.

Book_ID	Title	Author	Genre	Published_Year	Price	Stock
240	Stand-alone content-based hub	Lisa Ellis	Fantasy	1957	49.9	41
462	Innovative 3rdgeneration database	Allison Contreras	Fantasy	1988	49.23	62
238	Optimized even-keeled analyzer	Sherri Griffith	Fantasy	1975	48.97	72

Query 17: Retrieve the total quantity of books sold by each author

SQL Solution

```
SELECT b.author,SUM(o.quantity)AS Total_Books_Sold
FROM orders o
JOIN books b ON o.book_id=b.book_id
GROUP BY b.Author;
```

Explanation

Joins orders to books and sums the Quantity for each author.

Query Output

Rows returned: **314**. Preview below shows the first 8 rows.

Author	Total_Books_Sold
Aaron Hall Jr.	4
Aaron Rangel	6
Abigail Madden	5
Adrian Avery	6
Alexander Bailey	6
Alexander Smith	7
Alexander Wallace	20
Alexandra Cox	8

Query 18: Lists the cities where customers who spent over $30 are located

SQL Solution

```
SELECT DISTINCT c.city,total_amount
FROM orders o
JOIN customers c ON o.customer_id=c.customer_id
WHERE o.total_amount>30;
```

Explanation

Returns distinct City and Total_Amount pairs for individual orders over 30. Note: this does not calculate each city's total spending.

Query Output

Rows returned: **443**. Preview below shows the first 8 rows.

City	Total_Amount
Lake Paul	188.56
North Keith	216.6
Kelseyfort	85.5
East David	301.21
Richardsonville	136.36
Ramosstad	249.4
Rogersborough	82.92
New Carlosbury	144.84

Query 19: Find the customers who spent the most on orders

SQL Solution

```
SELECT c.customer_id,c.name,SUM(o.total_amount)AS Total_Spent
FROM orders o
JOIN customers c ON o.customer_id=c.customer_id
GROUP BY c.customer_id,c.name
ORDER BY Total_Spent DESC;
```

Explanation

Sums order amounts per customer and sorts customers by total spending, highest first.

Query Output

Rows returned: **307**. Preview below shows the first 8 rows.

Customer_ID	Name	Total_Spent
457	Kim Turner	1398.9
174	Jonathon Strickland	1080.95
364	Carrie Perez	1052.27
405	Julie Smith	991
386	Pamela Gordon	986.3
425	Ashley Perez	942.62
474	Anthony Young	929.19
163	Robert Clark	746.65

Query 20: Calculate the stock remaining after fulfilling all orders

SQL Solution

```
SELECT b.book_id,b.title,b.stock,COALESCE(SUM(o.quantity),0)AS Order_quantity,
b.stock-COALESCE(SUM(o.quantity),0)AS Remaining_Quantity
FROM books b
LEFT JOIN orders o ON b.book_id=o.book_id
GROUP BY b.book_id
ORDER BY b.book_id;
```

Explanation

Shows each book's recorded stock, sums its ordered quantity, and subtracts that quantity to estimate remaining stock.

Query Output

Rows returned: **500**. Preview below shows the first 8 rows.

Book_ID	Title	Stock	Order_quantity	Remaining_Quantity
1	Configurable modular throughput	100	3	97
2	Persevering reciprocal knowledge user	19	0	19
3	Streamlined coherent initiative	27	5	22
4	Customizable 24hour product	8	0	8
5	Adaptive 5thgeneration encoding	16	8	8
6	Advanced encompassing implementation	2	0	2
7	Open-architected exuding structure	95	5	90
8	Persistent local encoding	84	3	81

Conclusion

This project demonstrates practical SQL skills using a bookstore-style relational dataset. The 20 queries practise
SELECT, WHERE, DISTINCT, ORDER BY, LIMIT, aggregate functions, JOIN, LEFT JOIN, GROUP BY, HAVING, and COALESCE. The query results can be used to explore inventory levels, customer order activity, book demand, and recorded revenue.